МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных Технологий
Кафедра Программной инженерии
Специальность 6-05-0612-01 Программная инженерия.
Профилизация: Программирование обеспечение информационных технологий

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОМУ ПРОЕКТУ НА ТЕМУ:

«Разработка компилятора IAS-2025»

Выполнил студент Ильинковский Арсений Станиславович
(Ф.И.О.)

Руководитель проекта преп-ст. Гурина К. С.
(учен. степень, звание, должность, подпись, Ф.И.О.)

Заведующий кафедрой к.т.н. доц. Смелов В. В
(учен. степень, звание, должность, подпись, Ф.И.О.)

Консультант преп-ст. Гурина К. С.
(учен. степень, звание, должность, подпись, Ф.И.О.)

Курсовой проект защищен с оценкой _____

Минск 2025

Содержание

1. Спецификация языка программирования 6
1.1 Характеристика языка программирования 6
1.2 Алфавит языка 6
1.3 Применяемые сепараторы 6
1.4 Применяемые кодировки 6
1.5 Типы данных 7
1.6 Преобразование типов данных 8
1.7 Идентификаторы 8
1.8 Литералы 8
1.9 Объявления данных и область видимости 9
1.10 Инициализация данных 9
1.11 Инструкции языка 10
1.12 Операции языка 10
1.13 Выражения и их вычисления 11
1.14 Программные конструкции языка 11
1.15 Область видимости идентификаторов 12
1.16 Семантические проверки 12
1.17 Распределение оперативной памяти на этапе выполнения 13
1.18 Стандартные библиотеки и их состав 13
1.19 Ввод и вывод данных 14
1.20 Точка входа 14
1.21 Препроцессор 14
1.22 Соглашения о вызовах 14
1.23 Объектный код 14
1.25 Контрольный пример 15
2. Структура транслятора 16
2.1 Компоненты транслятора, их назначение и принципы взаимодействия 16
2.2 Перечень входных параметров транслятора 17
2.3 Перечень протоколов, формируемых транслятором и их содержимое 18
3. Разработка лексического анализатора 19
3.1 Структура лексического анализатора 19
3.2. Контроль входных символов 20
3.3 Удаление избыточных символов 20
3.4 Перечень ключевых слов 21
3.5 Основные структуры данных 23
3.6 Принцип обработки ошибок 25
3.7 Структура и перечень сообщений лексического анализатора 25
3.8 Параметры лексического анализатора 26
3.9 Алгоритм лексического анализа 26
3.10 Контрольный пример 26
4. Разработка синтаксического анализатора 27
4.1 Структура синтаксического анализатора 27

4.2 Контекстно-свободная грамматика, описывающая синтаксис языка 27
4.3 Построение конечного магазинного автомата..... 30
4.4 Основные структуры данных 31
4.5 Описание алгоритма синтаксического разбора 31
4.6 Структура и перечень сообщений синтаксического анализатора 31
4.7. Параметры синтаксического анализатора и режимы его работы..... 32
4.8. Принцип обработки ошибок..... 32
4.9. Контрольный пример 32
5. Разработка семантического анализатора..... 33
5.1 Структура семантического анализатора..... 33
5.2 Функции семантического анализатора..... 33
5.3 Структура и перечень сообщений семантического анализатора..... 33
5.4 Принцип обработки ошибок..... 34
5.5 Контрольный пример 34
6. Вычисление выражений 35
6.1 Выражения, допускаемые языком 35
6.2 Польская запись и принцип её построения..... 35
6.3 Программная реализация обработки выражений..... 36
6.4 Контрольный пример 36
7. Генерация кода..... 37
7.1 Структура генератора кода 37
7.2 Представление типов данных в оперативной памяти..... 37
7.3 Статическая библиотека..... 38
7.4 Особенности алгоритма генерации кода..... 39
7.5 Входные параметры генератора кода 39
7.6 Контрольный пример 39
8. Тестирование транслятора..... 41
8.1 Тестирование проверки на допустимость символов..... 41
8.2 Тестирование лексического анализатора 41
8.3 Тестирование синтаксического анализатора 42
8.4 Тестирование семантического анализатора..... 43
Заключение..... 45
Список использованных источников..... 46
Приложение А 47
Приложение Б..... 49
Приложение В 52
Приложение Г..... 62

Введение

В данном курсовом проекте поставлена задача разработки собственного языка программирования и транслятора для него. Название языка – IAS-2025. Написание транслятора будет осуществляться на языке C++, при этом код на языке IAS-2025 будет транслироваться в язык ассемблера.

Задание на курсовой проект можно разделить на следующие задачи:

– разработка спецификации языка IAS-2025;

– разработка лексического анализатора;

– разработка синтаксического анализатора;

– разработка семантического анализатора;

– разбор арифметических выражений;

– разработка генератора кода;

– тестирование транслятора;

Каждый из перечисленных этапов будет подробно рассмотрен в соответствующих разделах работы, начиная с описания синтаксиса языка IAS-2025 и заканчивая генерацией в целевой язык Assembler .

1. Спецификация языка программирования

1.1 Характеристика языка программирования

Язык программирования IAS-2025 является процедурным, строго типизируемым, транслируемым на язык assembler.

1.2 Алфавит языка

Алфавит IAS-2025 состоит из символов таблицы Windows-1251. Допустимо использование символов латинского алфавита (A…z), кириллицы (А…я). Запрещается использование служебных символов, а также некоторых символов. Например, использование знака тильды в IAS-2025 не предусмотрено.

1.3 Применяемые сепараторы

Сепараторы служат для разделения операций языка. Сепараторы, применяемые в языке программирования IAS-2025, приведены в таблице 1.1.

Таблица 1.1 - Применяемые сепараторы

Сепаратор	Назначение сепаратора
;	Разделитель инструкций
{ }	Программный блок
( )	Параметры, приоритетность операций в выражениях
пробел	Допускается везде, кроме идентификаторов и ключевых слов. Служит для их разделения
,	Разделитель параметров в функции

Сепараторы – программные конструкции, позволяющие разделять программные блоки кода.

1.4 Применяемые кодировки

Кодировка, используемая для написания программ на языке IAS-2025 - стандартная кодировка Windows-1251, представленная на Рисунке 1.1.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0				©	Ё	§	Є	·		°						
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1																
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
2		!	"	#	$	%	&		(	)	*	+				/
	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}]	~	□
	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127
8	Ђ	Ѓ	‚	ѓ	„	…	†	‡		‰	Љ	‹	Њ	Ќ	Ћ	Џ
	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143
9	ђ	‘	’	“	”		–	—		™	љ	›	њ	ќ	ћ	џ
	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159
A		Ў	ў	J	¤	Ґ	¦	§	Ё	©	Є	«		-	®	Ї
	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175
B	°	±	І	і	ґ	µ		·	ё	№	є	»	j	S	s	ї
	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191
C	А	Б	В	Г	Д	Е	Ж	З	И	Й	К	Л	М	Н	О	П
	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207
D	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	▪ 223
E	а	б	в	г	д	е	ж	з	и	й	к	л	м	н	о	п
	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239
F	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я
	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255

Рисунок 1.1 Таблица кодировки Windows 1251

Windows 1251 – это 8-битная кодировка символов, предназначенная для покрытия языков, использующих кириллицу.

1.5 Типы данных

Допускается использование фундаментальных типов данных, определенных в таблице 1.2.

Таблица 1.2 – Фундаментальные типы данных

Тип данных	Описание
int	Фундаментальный тип данных. Предусмотрен для объявления беззнаковых целочисленных данных (4 байта). Максимальное значение: 2147483647, минимальное значение: 0. Автоматически инициализируется нулевым значением.
str	Фундаментальный тип данных. Предусмотрен для объявления строк. (1 символ – 1 байт). Автоматическая инициализация строкой нулевой длины. Максимальное количество символов в строке – 255. Первый байт – длина строки.

Продолжение таблицы 1.2

char	Фундаментальный тип данных. Предусмотрен для объявления символов (1 байт).
bool	Фундаментальный тип данных. Используется для объявления булевых значений (true или false). Автоматически инициализируется значением false.

Эти типы данных позволяют выполнять множество операций и делают функционал языка шире

1.6 Преобразование типов данных

Преобразование типов данных в языке IAS-2025 не предусмотрено. Попытка преобразования типов данных приведет к ошибке.

1.7 Идентификаторы

Идентификаторы могут выступать в качестве имен функций, параметров, переменных. Зарезервированные идентификаторы не предусмотрены. Идентификаторы не должны совпадать с ключевыми словами.

Имя идентификатора составляется по следующим правилам:

– состоит из символов [a..z] и [A..Z] и '_';

– длина идентификатора не должна превышать 20 символов. При превышении максимально допустимой длины применяется усечение.

Рекомендуется использовать осмысленные имена идентификаторов, отражающие назначение переменной или функции, что повышает читаемость программного кода.

1.8 Литералы

В языке программирования IAS-2025 предусмотрены целочисленные и строковые литералы. Все литералы являются rvalue. Их правила записи приведены в таблице 1.3.

Таблица 1.3 – Правила записи литералов

Тип литерала	Описание литерала
Символьный	Символ, заключенный в ' .. ' (одинарные кавычки).
Строковый	Символы, заключенные в "…" (двойные кавычки), число которых не превышает 255.

Продолжение таблицы 1.3

Логический	Слова true или false, соответствующие истинности и лжи.
Целочисленный	IAS-2025 поддерживает представление литералов в следующих системах счисления: двоичная: [0, 1], начинается с префикса 'b'; восьмеричная: [0..7], начинается с префикса 'o'; десятичная: [0..9], без префикса; шестнадцатеричная: [0..9], [A..F], начинается с префикса 'h'. При выходе за пределы будет выброшена ошибка.

Литералы представляют неизменяемые значения. Литералы можно передавать переменным в качестве значения.

Rvalue – это выражение, которое не представляет собой объект, который занимает идентифицируемое место в памяти.

Значения целочисленных литералов, выходящих за пределы допустимых значений являются недопустимыми. Значения строковых литералов, у которых количество символов строки превышает 255 символов – автоматически урезается. Значения логических литералов, не соответствующих правилам, указанным выше, приводят к ошибке.

1.9 Объявления данных и область видимости

Для объявления переменной указывается тип данных, далее используется ключевое слово var, после чего имя идентификатора.

Для объявления функций используется ключевое слово function, перед которым указывается тип функции, а после – идентификатор.

Область видимости реализована Си-подобным образом: сверху-вниз. Все функции и процедуры имеют глобальную область видимости и могут быть объявлены только в глобальной области видимости. Переменные не могут быть глобальными. Любые идентификаторы должны быть объявлены до их использования.

1.10 Инициализация данных

Объектами-инициализаторами могут быть только идентификаторы или литералы. Также возможна инициализация непосредственно при объявлении переменной. Пример инициализации данных представлен в листинге 1.1.

```
int var b = h32;
str var stringOne = "Hello, my name is Arseniy";
```

Листинг 1.1 – Инициализация данных в языке IAS-2025

При объявлении предусмотрены значения по умолчанию: значение 0 для типа int, строка длины 0 ("") для типа str, а также значение false для булевого типа данных.

1.11 Инструкции языка

В языке программирования IAS-2025 применяются инструкции, представленные в таблице 1.4.

Таблица 1.4 – Инструкции языка

Инструкция языка	Синтаксис
Объявление переменной	<тип данных> var <идентификатор>; <тип данных>var<идентификатор> = <литерал>|<выражение>|<идентификатор>
Присваивание	<идентификатор> = <литерал>|<выражение>|<идентификатор>;
Вызов функций	<идентификатор функции> ([параметр 1] [, параметр 2]);
Вывод данных в консоль	output <литерал>|<идентификатор>;
Возврат из функции	return <литерал>|<идентификатор>;
Оператор цикла	repeat (<целочисленный литерал> | <выражение>){ <тело цикла>}
Условный оператор	if (<выражение>) then{ <тело истины>} else{<тело лжи>}

Блок для else в условном операторе необязателен.

1.12 Операции языка

Язык программирования IAS-2025 может выполнять арифметические операции, операции сравнения, побитовые операции, представленные в таблице 1.5.

Таблица 1.5 - Приоритетности операций языка программирования IAS-2025

Операция	Арифметическое назначение	Приоритетность операции
()	Приоритетность операций	1
+	Сложение	3
-	Вычитание	3
*	Умножение	4

Продолжение таблицы 1.5

>	Сравнение чисел	3
<	Сравнение чисел	3
^	Сравнение чисел	3
&	Сравнение чисел	3
<<	Сдвиг влево	5
>>	Сдвиг вправо	5
/	Деление	4
%	Взятие остатка от деления	4

Для повышения приоритетности следует использовать круглые скобки.

1.13 Выражения и их вычисления

Выражения относятся только к целому типу.

Предусмотрены следующие правила составления выражений:

– выражения записываются до ввода сепаратора «;»;

– выражение может содержать вызов функций;

– рассматриваются слева направо;

– для изменения приоритета операция используются круглые скобки ();

– реализация выражений происходит с помощью обратной польской записи.

Данные правила формируют основу для синтаксического и семантического анализа выражений, что позволяет обеспечить корректную обработку арифметических конструкций в языке IAS-2025.

1.14 Программные конструкции языка

Программные конструкции представлены в таблице 1.6.

Таблица 1.6 – Программные конструкции языка

Конструкция	Представление в языке

Главная функция	main
	{
	<инструкции языка>
	}

Цикл	repeat(<идентификатор1>|<литерал>
	<логический оператор><идентификатор2>|<литерал>)
	{
	<инструкции языка>
	}

Продолжение таблицы 1.6

Условный оператор	if (<идентификатор1>|<литерал> <логический оператор><идентификатор2>|<литерал>) then{<инструкции языка>} else{<инструкции языка>}

Функция	<тип данных> function <идентификатор>([<тип данных> param <идентификатор>], [<тип данных> param <идентификатор>])
	{
	<инструкции языка>
	return <идентификатор>|<литерал>
	}

Программные конструкции языка IAS-2025 представляют собой базовый функционал для выполнения различных операций, что делает возможным решать задачи различного уровня.

1.15 Область видимости идентификаторов

Все идентификаторы, объявленные внутри функции, являются локальными. Параметры видны только внутри функции, в которой объявлены. Глобальная область видимости не поддерживается.

1.16 Семантические проверки

Перечень семантических проверок, предусмотренных языком:
– деление на ноль;
– совпадение типов данных;
– корректность строковых выражений справа от знака равно;
– проверка на возвращаемое значение из функции;
– передаваемые параметры в функции при вызове;
– корректность условного выражения;
– переопределение типов данных;
– выход за пределы области видимости;
– усечение слишком длинного значения str-литерала;
– точка входа main;
– проверка типа идентификаторов;
– проверка лексем

Если ошибка возникает на этапе лексического анализа, синтаксический анализ не выполняется. В случае, когда три подряд идущие фразы не удаётся разобрать, работа транслятора прекращается. Если же ошибка появляется в процессе синтаксического анализа, проблемная фраза игнорируется

(считается отсутствующей), после чего выполняется попытка разобрать следующую фразу. Границей фразы считается точка с запятой.

Назначение семантического анализа – проверка смысловой правильности конструкций языка программирования.

1.17 Распределение оперативной памяти на этапе выполнения

Транслированный код использует две области памяти. В сегмент констант заносятся все литералы. В сегмент данных заносятся переменные, параметры функций. Локальная область видимости в исходном коде определяется за счет использования переменной, хранящей имя родительского блока.

1.18 Стандартные библиотеки и их состав

В языке IAS-2025 присутствует стандартная статическая библиотека: StandLib, подключается автоматически при трансляции кода в ассемблер. Функции стандартной библиотеки и их описание представлено в таблице 1.7.

Таблица 1.7 - Библиотека StandartLib языка IAS-2025

Имя функции	Возвращаемое значение	Принимаемые параметры	Описание
length	integer	str parm1 - строка	Функция вычисляет длину строки parm1
copy	string	str parm1 – строка str parm2 – строка int count - цифра	Функция выполняет копирование определенного количества символов из parm2 в строку parm1
output	отсутствует	integer <идентификатор> | <целочисленный литерал> <строковый литерал>	Функция выводит на консоль <выражение>
random	integer	int start – цифра int end – цифра	Функция генерирует случайное числовое значение в диапазоне от start до end
factorialOfNumber	integer	int num - цифра	Функция возвращает факториал заданного числа num

Продолжение таблицы 1.7

squareOfNumber	integer	int num - цифра	Функция извлекает квадратный корень из num и возвращает значение
asciiCode	integer	char ch - символ	Фукнция вовзращает код получаемого символа в кодировке ASCII
getLocalTimeAndDate	string	Без параметров	Функция возвращает текущее локальное время и дату

Стандартная библиотека IAS-2025 содержит множество функций, использование которых упрощает написание кода.

1.19 Ввод и вывод данных

Ввод данных в языке IAS-2025 не предусмотрен. Вывод данных осуществляется с помощью ключевого слова «output» Есть возможность выводить значение переменных и строк, которые будут заключены в двойные кавычки.

1.20 Точка входа

Точкой входа в программе является ключевое слово «main». Точка входа не может отсутствовать или быть переопределена.

1.21 Препроцессор

Препроцессор в языке программирования IAS-2025 не предусмотрен.

1.22 Соглашения о вызовах

Используется соглашение stdcall, все параметры передаются в стек справа налево, память освобождается вызываемым кодом.

1.23 Объектный код

Язык IAS-2025 транслируется в исходный код для языка ассемблера.

1.24 Классификация сообщений транслятора

Классификация ошибок представлена в таблице 1.8.

Таблица 1.8 - Классификация ошибок

Номера ошибок	Характеристика
0 – 99	Системные ошибки
100 – 104	Ошибки входных параметров
105 – 109	Ошибки при открытии файла
110 – 119	Ошибки при чтении файла
120 – 140	Ошибки лексического анализа
600 – 610	Ошибки синтаксического анализа
700 – 750	Ошибки семантического анализа

Обрабатываются ошибки на всех этапах обработки исходного кода, то есть во время прохождения различных этапов анализа.

1.25 Контрольный пример

Контрольный пример демонстрирует некоторые особенности языка IAS-2025: его фундаментальные типы, основные структуры, использование пользовательских функций, использование функций статической библиотеки, программные конструкции языка. Исходный код контрольного примера представлен в приложении А.

2. Структура транслятора

2.1 Компоненты транслятора, их назначение и принципы взаимодействия

В языке IAS-2025 исходный код транслируется в язык Assembler. Транслятор языка разделён на отдельные части, которые взаимодействуют между собой и выполняют отведённые им функции, которые представлены в пункте 2.1. Для того чтобы получить ассемблерный код, используется выходные данные работы лексического анализатора, а именно таблица лексем и таблица идентификаторов. Для указания выходных файлов используются входные параметры транслятора, которые описаны в таблице 2.1. Структура транслятора языка IAS-2025 приведена на рисунке 1.

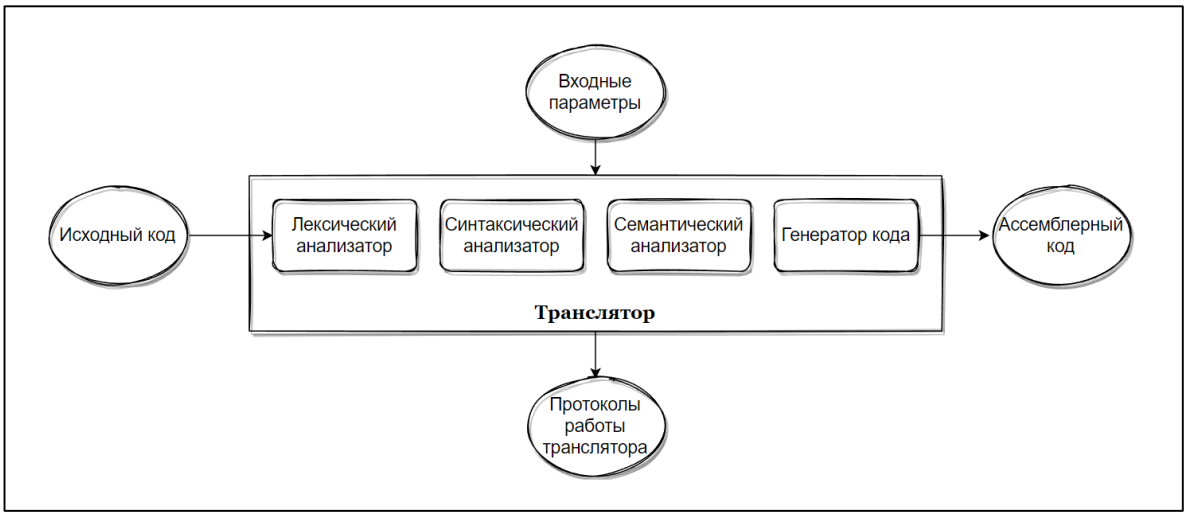

Рисунок 2.1 – Структура транслятора языка IAS-2025

Первая стадия работы компилятора называется лексическим анализом, а программа, её реализующая, – лексическим анализатором (лексер). На вход лексического анализатора подаётся последовательность символов входного языка. Он производит предварительный разбор текста, преобразующий единый массив текстовых символов в массив отдельных слов (в теории компиляции вместо термина «слово» часто используют термин «токен»). Примеры лексических единиц: идентификаторы, числа, символы операций, служебные слова и т.д. Лексический анализатор преобразует исходный текст, заменяя лексические единицы их внутренним представлением – лексемами, для создания промежуточного представления исходной программы. Каждой лексеме сопоставляется ее тип и запись в таблице идентификаторов, в которой хранится дополнительная информация. Таблица лексем (ТЛ) и таблица идентификаторов (ТИ) являются входом для следующей фазы компилятора – синтаксического анализа.

Лексический анализатор, последовательно обрабатывая входной поток символов, выделяя из него значимые элементы языка и подготавливая их к дальнейшим этапам трансляции, выполняет следующие функции:

− убрать все лишние пробелы;

− выполнить распознавание лексем;

− построить таблицу лексем и таблицу идентификаторов;

− при неуспешном распознавании или обнаружении некоторых ошибок во входном тексте выдать сообщение об ошибке.

Синтаксический анализатор – часть компилятора, выполняющая синтаксический анализ, то есть проверку исходного кода на соответствие правилам грамматики. Входной информацией для синтаксического анализа является таблица лексем и таблица идентификаторов. Выходной информацией является дерево разбора

Семантический анализатор – часть транслятора, выполняющая семантический анализ, то есть проверку исходного кода на наличие ошибок, которые невозможно отследить при помощи регулярной и контекстно-свободной грамматики. Входными данными являются таблица лексем и идентификаторов.

Генератор кода – часть транслятора, выполняющая генерацию ассемблерного кода на основе полученных данных на предыдущих этапах трансляции. На вход генератора подаются таблица лексем и таблица идентификаторов, на основе которых генерируется файл с ассемблерным кодом.

2.2 Перечень входных параметров транслятора

Для формирования файлов с результатами работы лексического, синтаксического и семантического анализаторов используются входные параметры транслятора, которые приведены в таблице 2.1.

Таблица 2.1 - Входные параметры транслятора языка IAS-2025

Входной параметр	Описание параметра	Значение по умолчанию
-in:<путь к in-файлу>	Файл с исходным кодом на языке IAS-2025 , имеющий расширение .txt	Не предусмотрено
-log:<путь к log-файлу>	Файл журнала для вывода протоколов работы программы.	Значение по умолчанию: <имя in-файла>.log

Продолжение таблицы 2.1

-out:<путь к out-файлу>	Выходной файл – результат работы транслятора. Содержит исходный код на языке асемблера.	Значение по умолчанию: <имя in-файла>.asm

Входные параметры указываются через командную строку вручную.

2.3 Перечень протоколов, формируемых транслятором и их содержимое

В ходе работы программы формируются протоколы работы лексического, синтаксического и семантического анализаторов, которые содержат в себе перечень протоколов работы. В таблице 2.2 приведены протоколы, формируемые транслятором и их содержимое.

Таблица 2.2 - Протоколы, формируемые транслятором языка IAS-2025

Формируемый протокол	Описание выходного протокола
Файл журнала, заданный параметром "-log:"	Файл с протоколом работы транслятора языка программирования IAS-2025. Содержит информацию про входные параметры, общем количестве символов и строк(исходные данные), протокол работы синтаксического анализатора, полученный на этапе синтаксического анализа.
Выходной файл с названием "Table.id.txt"	Файл содержит таблицу идентификаторов, сформированную во время лексического анализа.
Выходной файл с названием "Table.lex.txt"	Файл содержит таблицу лексем, сформированную во время лексического анализа.
Выходной файл, заданный параметром "-out:"	Результат работы программы – файл, содержащий исходный код на языке ассемблера.

Все файлы создаются в каталоге, относительно исполняемого модуля.

3. Разработка лексического анализатора

3.1 Структура лексического анализатора

Первая стадия работы компилятора называется лексическим анализом, а программа, её реализующая, – лексическим анализатором (лексером). На вход лексического анализатора подаётся исходный код входного языка. Лексический анализатор выделяет в этой последовательности простейшие конструкции языка. Лексический анализатор производит предварительный разбор текста, преобразующий единый массив текстовых символов в массив токенов.

Примеры лексических единиц: идентификаторы, числа, символы операций, служебные слова и т.д. Лексический анализатор преобразует исходный текст, заменяя лексические единицы их внутренним представлением – лексемами, для создания промежуточного представления исходной программы. Каждой лексеме сопоставляется ее тип и запись в таблице идентификаторов, в которой хранится дополнительная информация.

Функции лексического анализатора:

– удаление «пустых» символов и комментариев. Если «пустые» символы (пробелы, знаки табуляции и перехода на новую строку) и комментарии будут удалены лексическим анализатором, синтаксический анализатор никогда не столкнется с ними (альтернативный способ, состоящий в модификации грамматики для включения «пустых» символов и комментариев в синтаксис, достаточно сложен для реализации);

– распознавание идентификаторов и ключевых слов;

– распознавание констант;

– распознавание разделителей и знаков операций.

Исходный код программы представлен в приложении А, структура лексического анализатора представлена на рисунке 3.1.

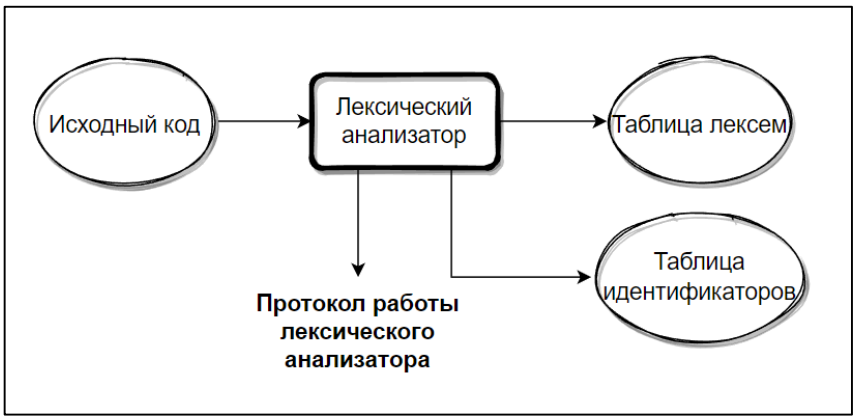

Рисунок 3.1 – Структура лексического анализатора

Лексический анализ служит фундаментом для последующих этапов компиляции, обеспечивая синтаксическому анализатору корректную и

структурированную последовательность токенов. Качество работы лексера напрямую влияет на точность разбора и дальнейшую обработку программы.

3.2. Контроль входных символов

Для удобной работы с исходным кодом, при передаче его в лексический анализатор, все символы разделяются по категориям. Таблица входных символов представлена на рисунке 3.2.

```
IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::S,   '|', IN::F, IN::F, IN::F, IN::F, IN::F,`
IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F,`
IN::S, IN::F, IN::T, IN::V, IN::F, IN::V, IN::V, IN::T, IN::V, IN::V, IN::V, IN::V, IN::V, IN::V, IN::F, IN::V,`
IN::N, IN::N, IN::N, IN::N, IN::N, IN::N, IN::N, IN::N, IN::N, IN::N, IN::T, IN::V, IN::T, IN::V, IN::T, IN::F,`
IN::F, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T,`
IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::F, IN::F, IN::F, IN::V, IN::T,`
IN::F, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T,`
IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::V, IN::F, IN::V, IN::F, IN::F,`
                                                                                                              `

IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F,`
IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::T, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F,`
IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F,`
IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F,`
IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T,`
IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T,`
IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T,`
IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T,`
```

Рисунок 3.2 – Таблица контроля входных символов

Категории входных символов представлены в таблице 3.1.

Таблица 3.1 - Соответствие символов и их значений в таблице

Значение в таблице входных символов	Символы
Разрешенный	T
Запрещенный	F
Игнорируемый	I
Пробел, табуляция	S
Операторы	V
Цифры	N

Также лексический анализатор осуществляет контроль допустимости входных символов, классифицируя их по типам: разрешённые, запрещённые, игнорируемые и т.д. Это позволяет заранее исключить недопустимые символы и упростить дальнейшую обработку исходного кода.

3.3 Удаление избыточных символов

Избыточными символами являются символы табуляции и пробелы.

Избыточные символы удаляются на этапе разбиения исходного кода на токены.

Описание алгоритма удаления избыточных символов:

– посимвольно считываем файл с исходным кодом программы;

– встреча пробела или знака табуляции является своего рода встречей символа-сепаратора;

– в отличие от других символов-сепараторов не записываем в очередь лексем эти символы, т.е. игнорируем.

Удаление избыточных символов позволяет упростить структуру входного потока и избежать обработки нерелевантных данных на последующих этапах компиляции. Такой подход повышает эффективность лексического анализа и снижает вероятность синтаксических ошибок, связанных с лишними пробелами или табуляцией.

3.4 Перечень ключевых слов

Лексический анализатор преобразует исходный текст, заменяя лексические единицы лексемами для создания промежуточного представления исходной программы. Соответствие токенов и лексем приведено в таблице 3.2.

Таблица 3.2 - Соответствие токенов и сепараторов с лексемами

Токен	Лексема	Пояснение
int, string	t	Названия строковых и целочисленных типов языка.
char	c	Название символьного типа данных.
bool	x	Название логического типа данных.
Идентификатор	i	Длина идентификатора – 20 символов.
Литерал	l	Литерал строкового и целочисленного типа данных.
Литерал	k	Литерал для символьного типа данных.
Литерал	o	Литерал для логического типа данных
function	f	Объявление функции.
var	v	Объявление переменной
param	p	Объявление параметра функции
return	r	Выход из функции/процедуры.
main	m	Главная функция.
output	w	Вывод данных
if	?	Уловный оператор.
then	:	Истинная ветвь условного оператора.
else	!	Ложная ветвь условного оператора.
repeat	~	Указывает на начало тела цикла.
;	;	Разделение выражений.
,	,	Разделение параметров функций.

Продолжение таблицы 3.2

)	)	Закрытие блока для передачи параметров, приоритет операций.
=	=	Знак присваивания.
+ - * / %	 # 	Знаки операций.
>< ^ &	>< ^ &	Знаки логических операторов.
<< >>	b n	Знаки побитовых сдвигов.
#	Нет	Комментарии.
(	(	Передача параметров в функцию, приоритет операций.
{	{	Начало блока/тела функции.
}	}	Закрытие блока/тела функции.

Пример реализации таблицы лексем представлен в приложении Б.

Каждому выражению соответствует детерминированный конечный автомат, по которому происходит разбор данного выражения. На каждый автомат в массиве подаётся токен и с помощью регулярного выражения, соответствующего данному графу переходов, происходит разбор. В случае успешного разбора выражения оно записывается в таблицу лексем. Если выражение является идентификатором или литералом, информация также заносится в таблицу идентификаторов. Дополнительно использование единой таблицы ключевых слов обеспечивает однозначность интерпретации конструкций языка и исключает неоднозначности при разборе. Благодаря применению детерминированных конечных автоматов процесс лексического анализа становится формально описанным, воспроизводимым и легко расширяемым при добавлении новых элементов языка. Такой подход обеспечивает строгий контроль корректности входных данных и позволяет своевременно выявлять лексические ошибки ещё до этапа синтаксического анализа. В результате повышается надёжность всей системы компиляции и упрощается последующая обработка программы. Структура конечного автомата изображена на рисунке 3.3.

```
RELATION::RELATION(char c = 0x00, short ns = NULL)
{
    symbol = c;
    nnode = ns;
};

NODE::NODE()                                    // по умолчанию
{
    n_relation = 0;
    RELATION* relations = NULL;
};

NODE::NODE(short n, RELATION rel, ...)          // с параметрами
{
    n_relation = n;
    RELATION* ptr = &rel;
    relations = new RELATION[n];
    for (short i = 0; i < n; i++)
        relations[i] = ptr[i];
};

FST::FST(const char* s, const char lex, short ns, NODE n, ...)
{
    lexema = lex;
    string = s;
    nstates = ns;
    nodes = new NODE[ns];
    NODE* ptr = &n;
    for (int k = 0; k < ns; k++)
        nodes[k] = ptr[k];
    rstates = new short[nstates];
    memset(rstates, 0xff, sizeof(short) * nstates);
    rstates[0] = 0;
    position = -1;
```

Рисунок 3.3 – Структура конечного автомата

Пример графа перехода конечного автомата изображен на рисунке 3.4.

```
ST::NODE(1, FST::RELATION('o', 1)), \
ST::NODE(1, FST::RELATION('u', 2)), \
ST::NODE(1, FST::RELATION('t', 3)), \
ST::NODE(1, FST::RELATION('p', 4)), \
ST::NODE(1, FST::RELATION('u', 5)), \
ST::NODE(1, FST::RELATION('t', 6)), \
```

Рисунок 3.4 – Пример реализации графа КА для токена output

3.5 Основные структуры данных

Основными структурами данных лексического анализатора являются таблица лексем и таблица идентификаторов. Таблица лексем представляет собой последовательность записей, каждая из которых содержит номер лексемы, саму лексему (lexema), полученную в результате разбиения исходного кода, знак оператора (sign), номер строки в исходном файле (sn), номер токена (st), а также индекс в таблице идентификаторов (idxTI), если

текущая лексема относится к идентификаторам. Такая структура позволяет не только хранить результаты лексического анализа, но и обеспечивать удобный доступ к информации на последующих этапах компиляции, включая синтаксический и семантический анализ. Таблица идентификаторов, в свою очередь, содержит дополнительные сведения об именах переменных, функций и других сущностей, что делает её важным элементом для проверки корректности программы. Код C++ со структурой таблицы лексем представлен на рисунке 3.5.

```
struct Entry                        // строка таблицы лексем
{
    char lexema;                    // лексема
    char sign;                      // знак лексемы v или LT_V_NULL
    int sn;                         // номер строки в исходном тексте
    int tn;                         // номер токена в строке
    int idxTI;                      // индекс в таблице идентификаторов или LT_TI_NULLIDX

    Entry(char lexema, int sn, int tn, int idxTI);      // для идентификатор
    Entry(char lexema, char sign, int sn, int tn);      // для операторов
    Entry(char lexema, int sn, int tn);                 // остальные лексемы
    Entry(char lexema);
};

struct LexTable                     // экземпляр таблицы лексем
{
    int maxsize;                    // емкость таблицы лексем < LT_MAXSIZE
    int current_size;               // текущий размер таблицы лексем < maxsize
    Entry** table;                  // массив указателей на строки таблицы лексем

    LexTable();
    LexTable(int size);         // создать таблицу лексем
};
void Add(LexTable& lextable, Entry* entry);                     // добавление лексем
Entry GetEntry(LexTable& lextable, int n);                      // получить строку таблицы лексем
void PrintLexTable(LexTable& lextable, const wchar_t* in);      // вывод лексем в файл
void Delete(LexTable& lextable);                                // удалить таблицу лексем (освободить память
```

Рисунок 3.5 – Структура таблицы лексем

Таблица идентификаторов содержит имя идентификатора (id), номер в таблице лексем (idxfirstLE), тип данных (iddatatype), тип идентификатора (idtype) и значение (или параметры функций) (value). Код C++ со структурой таблицы идентификаторов представлен на рисунке 3.6.

```
enum class IDDATATYPE { DEF, INT, STR, CHAR, BOOL};                 // типы данных идентификаторов: не определен, integer, string
enum class IDTYPE { D, V, F, P, L, C };                     // типы идентификаторов: не определен, переменная, функция, параметр, литер

struct Entry    // строка таблицы идентификаторов
{
    int idxfirstLE;                     // индекс первой строки в таблице лексем
    char areaOfVisibility[ID_MAXSIZE]{};    // область видимости
    char id[ID_MAXSIZE]{};              // идентификатор (автоматически усекается до ID_MAXSIZE)
    IDDATATYPE  iddatatype;             // тип данных
    IDTYPE  idtype;                     // тип идентификатора
    union
    {

        int vint;// значение integer
        bool vbool;
        struct
        {
            unsigned char len;          // количесво символов в string
            char str[TI_STR_MAXSIZE];   // символы string
        } vstr;                         // значение string
    }value;                             // значение идентификатора
    struct Param
    {
        int count;                      // количество параметров функции
        vector<IDDATATYPE> types;       // типы параметров функции
    } params;
    string FullName;
    Entry(int idxfirstLE, string areaOfVisibility, const char* id, IDDATATYPE iddatatype, IDTYPE idtype);
    Entry(int idxfirstLE, IDDATATYPE iddatatype, IDTYPE idtype, char* value);
};
struct IdTable // экземпляр таблицы идентификаторов
{
    int maxsize;            // емкость таблицы идентификаторов < TI_MAXSIZE
    int current_size;       // текущий размер таблицы идентификаторов < maxsize
    Entry** table;          // массив указателей на строки таблицы идентификаторов

    IdTable(int size);
```

Рисунок 3.6 – Структура таблицы идентификаторов

Совместное использование таблицы лексем и таблицы идентификаторов обеспечивает целостное и структурированное представление исходной программы, позволяя эффективно передавать результаты лексического анализа на последующие этапы обработки. Благодаря такой организации данных компилятор получает возможность быстро обращаться к необходимой информации, что повышает производительность и надёжность всего процесса трансляции.

3.6 Принцип обработки ошибок

Для обработки ошибок лексический анализатор использует таблицу с сообщениями. Структура сообщений содержит информацию о номере сообщения, номер строки и позицию, где было вызвано сообщение в исходном коде. Перечень сообщений представлен на рисунке 3.7.

```
IOR_ENTRY(1, "Системный сбой"),
IOR_ENTRY_NODEF(2), ERROR_ENTRY_NODEF(3), ERROR_ENTRY_NODEF(4), ERROR_ENTRY_NODEF(5),
IOR_ENTRY_NODEF(6), ERROR_ENTRY_NODEF(7), ERROR_ENTRY_NODEF(8), ERROR_ENTRY_NODEF(9),
IOR_ENTRY_NODEF10(10), ERROR_ENTRY_NODEF10(20), ERROR_ENTRY_NODEF10(30), ERROR_ENTRY_NODEF10(40), ERROR_ENTRY_NODEF10(50),
IOR_ENTRY_NODEF10(60), ERROR_ENTRY_NODEF10(70), ERROR_ENTRY_NODEF10(80), ERROR_ENTRY_NODEF10(90),
IOR_ENTRY(100, "Параметр -in должен быть задан"),
IOR_ENTRY(101, "Превышена длина входного параметра"),
IOR_ENTRY_NODEF(102), ERROR_ENTRY_NODEF(103), ERROR_ENTRY_NODEF(104),
IOR_ENTRY(105, "Ошибка при открытии файла с исходным кодом(-in)"),
IOR_ENTRY(106, "Ошибка при создании файла протокола (-log)"),
IOR_ENTRY(107, "Ошибка при создании файла протокола (-out)"),
IOR_ENTRY_NODEF(108), ERROR_ENTRY_NODEF(109),
IOR_ENTRY(110, "Недопустимый символ в исходном файле (-in)"),
IOR_ENTRY(111, "Нет закрывающей кавычки (-in)"),
IOR_ENTRY_NODEF(112), ERROR_ENTRY_NODEF(113), ERROR_ENTRY_NODEF(114), ERROR_ENTRY_NODEF(115),
IOR_ENTRY_NODEF(116), ERROR_ENTRY_NODEF(117), ERROR_ENTRY_NODEF(118), ERROR_ENTRY_NODEF(119),
IOR_ENTRY(120, "Нераспознанная лексема"),
IOR_ENTRY(121, "Тип данных идентификатора не определен"),
IOR_ENTRY(122, "Превышен размер таблицы лексем"),
IOR_ENTRY(123, "Таблица лексем переполнена"),
IOR_ENTRY(124, "Попытка обращения к незаполненной строке таблицы лексем"),
IOR_ENTRY(125, "Превышен размер лексемы"),
IOR_ENTRY(126, "Ошибка при открытии файла протокола (таблица лексем)"),
IOR_ENTRY(127, "Превышен размер таблицы идентификаторов"),
IOR_ENTRY(128, "Таблица идентификаторов переполнена"),
IOR_ENTRY(129, "Попытка обращения к незаполненной строке таблица идентификаторов"),
IOR_ENTRY(130, "Выход за пределы области видимости"),
IOR_ENTRY(131, "Переопределение"),
IOR_ENTRY(132, "Тип идентификатора не определен"),
IOR_ENTRY(133, "Отсутствует точка входа main"),
```

Рисунок 3.7 – Сообщения лексического анализатора

При возникновении ошибки, лексический анализатор выбрасывает исключение – работа программы останавливается.

3.7 Структура и перечень сообщений лексического анализатора

Ошибки, возникающие в процессе трансляции программы, фиксируются в протокол, заданный входным параметрами. В случае возникновения ошибок происходит их протоколирование с номером ошибки и диагностическим сообщением.

3.8 Параметры лексического анализатора

Результаты работы лексического анализатора, а именно таблицы лексем и идентификаторов выводятся в два отдельных файла – lextable.txt и idtable.txt.

3.9 Алгоритм лексического анализа

Алгоритм лексического анализа следующий:

– проверяет входной поток символов программы на исходном языке на допустимость, удаляет лишние пробелы и добавляет сепаратор для вычисления номера строки для каждой лексемы;

– для выделенной части входного потока выполняется функция распознавания лексемы;

– при успешном распознавании информация о выделенной лексеме заносится в таблицу лексем и таблицу идентификаторов, и алгоритм возвращается к первому этапу;

– формирует протокол работы;

– при неуспешном распознавании выдается сообщение об ошибке.

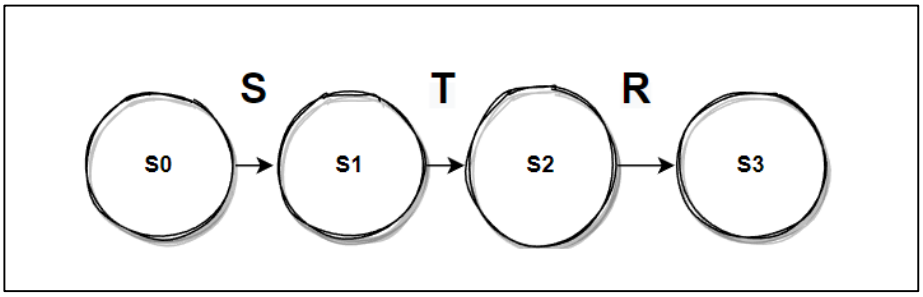

Рисунок 3.2 Пример графа переходов для цепочки **str**

Распознавание цепочек основывается на работе конечных автоматов. Работу конечного автомата можно проиллюстрировать с помощью графа переходов. Пример графа для цепочки «str» представлен на рисунке 3.2, где S0 – начальное, а S3 – конечное состояние автомата.

3.10 Контрольный пример

Результат работы лексического анализатора в виде таблиц лексем и идентификаторов, соответствующих контрольному примеру, представлен в приложении Б.

4. Разработка синтаксического анализатора

4.1 Структура синтаксического анализатора

Синтаксический анализатор: часть компилятора, выполняющая синтаксический анализ, то есть исходный код проверяется на соответствие правилам грамматики.

Описание структуры синтаксического анализатора языка представлено на рисунке 4.1.

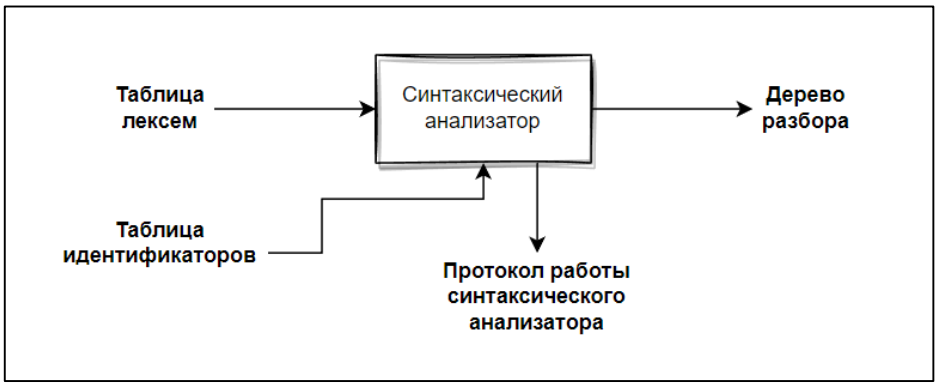

Рисунок 4.1 – Структура синтаксического анализатора

Входной информацией для синтаксического анализа является таблица лексем и таблица идентификаторов. Выходной информацией– дерево разбора

4.2 Контекстно-свободная грамматика, описывающая синтаксис языка

В синтаксическом анализаторе транслятора языка IAS-2025 используется контекстно-свободная грамматика $G = \langle T, N, P, S \rangle$, где

T – множество терминальных символов (было описано в разделе 1.2 данной пояснительной записки),

N – множество нетерминальных символов (первый столбец таблицы 4.1),

P – множество правил языка (второй столбец таблицы 4.1),

S – начальный символ грамматики, являющийся нетерминалом.

Эта грамматика имеет нормальную форму Грейбах, т.к. она не леворекурсивная (не содержит леворекурсивных правил)

Описание нетерминальных символов содержится в таблице 4.1.

Таблица 4.1 - Таблица правил переходов нетерминальных символов

Символ	Правила	Какие правила порождает
S	S-> m{N}S S-> tfi(F){NrU;}S	Стартовые правила, описывающее общую структуру программы

Продолжение таблицы 4.1

	S ->cfi(F){NrU;}S S ->xfi(F){NrU;}S S-> λ	
N	N-> tY;N N -> cY; N N -> xY; N N-> i=E;N N-> wE;N N-> ~KZN N -> ~(l)ZN N-> ?KJN N -> iM;N N-> λ	Операторы: объявления, присваивания, вызовы, условные и циклические конструкции
U	U -> l U -> k U -> i U -> lU U -> iU U -> i,U U -> l,U U -> k,U U -> #l U -> #k U -> #lU U -> #l,U	Только литерал или идентификатор (один или несколько)
J	J-> :{N} J-> :{N}!{N}	Правила условного перехода
Y	Y-> vi Y->vi=L Y -> vi=E M Y -> vi=k Y -> vi=o	Правило определения переменной
K	K -> (UU) K -> (U>U) K -> (U&U) K -> (U^U)	Правила определения условия перехода
E	E -> iM E -> lM E -> oM E -> kM E -> (E)M E -> i(W)M	Правила выражений

Продолжение таблицы 4.1

M	M -> #E	Правила арифметический операторов
	M -> bE M -> nE M -> λ	
F	F -> tP F -> cP F -> xP F -> tP,F F -> cP,F F -> xP,F	Правила определения параметров функции
P	P-> pi	Правило определения параметров функции
W	W -> i W -> l W -> k W -> o W -> i,W W -> l,W W -> k,W W -> o,W W -> λ	Правила вызова функции с параметрами
L	L -> i L -> l L -> k L -> o L -> i(U) L -> i() L -> #i L -> #l L -> #k L -> #o L -> #iL L -> #lL L -> #kL L -> #oL L -> iL L -> lL L -> oL L -> E M	Начальная инициализация
Z	Z->{N}	Тело цикла

На основе представленной грамматики синтаксический анализатор строит дерево разбора, последовательно применяя правила вывода к входной последовательности токенов. Такая формальная структура позволяет

однозначно интерпретировать каждую конструкцию языка IAS-2025 и обеспечивает корректный переход к этапу семантического анализа.

4.3 Построение конечного магазинного автомата

Конечный автомат с магазинной памятью представляет собой семерку $M = \langle Q, V, Z, \delta, q_0, z_0, F \rangle$. Подробное описание компонентов магазинного автомата представлено в таблице 4.2.

Таблица 4.2 – Описание компонентов магазинного автомата

Компонента	Определение	Описание
Q	Множество состояний автомата	Состояние автомата представляет из себя структуру, содержащую позицию на входной ленте, номера текущего правила и цепочки и стек автомата
V	Алфавит входных символов	Алфавит представляет из себя множества терминальных и нетерминальных символов, описание которых содержится в таблица 3.1 и 4.1.
Z	Алфавит специальных магазинных символов	Алфавит магазинных символов содержит стартовый символ и маркер дна стека (представляет из себя символ $)
δ	Функция переходов автомата	Функция представляет из себя множество правил грамматики, описанных в таблице 4.1.
q_0	Начальное состояние автомата	Состояние, которое приобретает автомат в начале своей работы. Представляется в виде стартового правила грамматики
z_0	Начальное состояние магазина автомата	Символ маркера дна стека $
F	Множество конечных состояний	Конечные состояние заставляют автомат прекратить свою работу. Конечным состоянием является пустой магазин автомата и совпадение позиции на входной ленте автомата с размером ленты

Совокупность перечисленных элементов определяет поведение магазинного автомата и задаёт правила обработки входной строки с учётом содержимого стека. Благодаря такой структуре автомат способен распознавать

контекстно-свободные языки, что делает его подходящим инструментом для реализации синтаксического анализа в трансляторе.

4.4 Основные структуры данных

Основные структуры данных синтаксического анализатора представляются в виде структуры магазинного конечного автомата, выполняющего разбор исходной ленты, и структуры грамматики Грейбах, описывающей синтаксические правила языка IAS-2025. Данные структуры в приложении В.

4.5 Описание алгоритма синтаксического разбора

Принцип работы автомата, следующий:

– в магазин записывается стартовый символ;

– на основе полученных ранее таблиц формируется входная лента;

– запускается автомат;

– выбирается цепочка, соответствующая нетерминальному символу, записывается в магазин в обратном порядке;

– если терминалы в стеке и в ленте совпадают, то данный терминал удаляется из ленты и стека. Иначе возвращаемся в предыдущее сохраненное состояние и выбираем другую цепочку нетерминала;

– если в магазине встретился нетерминал, переходим к пункту 4;

– если наш символ достиг дна стека, и лента в этот момент пуста, то синтаксический анализ выполнен успешно. Иначе генерируется исключение.

Такой алгоритм обеспечивает детерминированный и воспроизводимый процесс разбора, позволяя однозначно установить корректность структуры входной программы.

4.6 Структура и перечень сообщений синтаксического анализатора

Перечень сообщений синтаксического анализатора представлен на рисунке 4.3.

```
ROR_ENTRY(601, "Ошибочный оператор"),
ROR_ENTRY(602, "Ошибка в выражении"),
ROR_ENTRY(603, "Ошибка в параметрах при определении функции"),
ROR_ENTRY(604, "Ошибка в параметрах при вызове функции"),
ROR_ENTRY(605, "Ошибка: можно использовать только литерал или идентификатор"),
ROR_ENTRY(606, "Ошибка при условном переходе"),
ROR_ENTRY(607, "Ошибка при определении переменной"),
ROR_ENTRY(608, "Ошибка при определении условия перехода"),
ROR_ENTRY(609, "Ошибка при начальной инициализации переменной"),
```

Рисунок 4.3 Сообщения синтаксического анализатора

На этапе синтаксического анализа определено 11 потенциально возможных ошибок, каждая из которых обрабатывается и в случае возникновения – срабатывает и выводится.

4.7. Параметры синтаксического анализатора и режимы его работы

Входной информацией для синтаксического анализатора является таблица лексем и идентификаторов. Кроме того, используется описание грамматики в форме Грейбах. Результаты работы лексического разбора, а именно дерево разбора и протокол работы автомата с магазинной памятью выводятся в журнал работы программы.

4.8. Принцип обработки ошибок

Синтаксический анализатор выполняет разбор исходной последовательности лексем до тех пор, пока не дойдёт до конца цепочки лексем или не найдёт ошибку. Тогда анализ останавливается и выводится сообщение об ошибке (если она найдена). Если в процессе анализа находятся более трёх ошибок, то анализ останавливается.

4.9. Контрольный пример

Результаты работы лексического разбора, а именно дерево разбора и протокол работы автомата с магазинной памятью приведены в приложении В.

5. Разработка семантического анализатора

5.1 Структура семантического анализатора

Семантический анализатор принимает на свой вход результаты работ лексического и синтаксического анализаторов, то есть таблицы лексем, идентификаторов и результат работы синтаксического анализатора, то есть дерево разбора, и последовательно ищет необходимые ошибки. Некоторые проверки (такие как проверка на единственность точки входа, проверка на предварительное объявление переменной) осуществляются в процессе лексического анализа. Общая структура обособленно работающего (не параллельно с лексическим анализом) семантического анализатора представлена на рисунке 5.1.

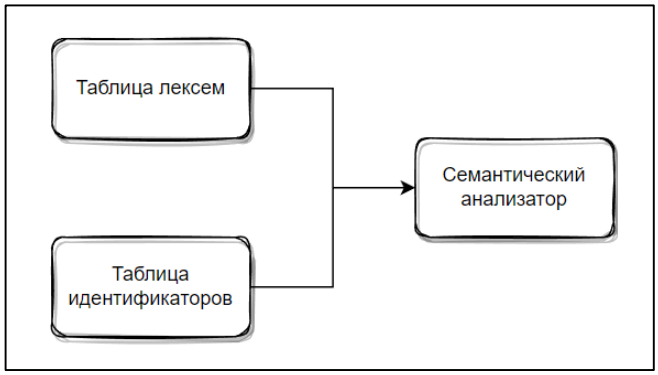

Рисунок 5.1 Структура семантического анализатора

5.2 Функции семантического анализатора

Семантический анализатор выполняет проверку на основные правила языка (семантики языка), которые описаны в разделе 1.16.

5.3 Структура и перечень сообщений семантического анализатора

Сообщения, формируемые семантическим анализатором, представлены на рисунке 5.1.

```
ROR_ENTRY(701, "Типы данных в выражении не совпадают"),
ROR_ENTRY(702, "Недопустимое строковое выражение справа от знака \'=\'"),
ROR_ENTRY(703, "Тип функции и возвращаемое значение не совпадают"),
ROR_ENTRY(704, "Несовпадение типов передаваемых параметров"),
ROR_ENTRY(705, "Количество ожидаемых функцией и передаваемых параметров не совпадают"),
ROR_ENTRY(706, "Неверное условное выражение"),
ROR_ENTRY_NODEF(707), ERROR_ENTRY_NODEF(708), ERROR_ENTRY_NODEF(709),
ROR_ENTRY(710, "Операция сдвига приводит к переполнению типа int"),
ROR_ENTRY(711, "Числовой литерал превышает допустимый диапазон типа int"),
ROR_ENTRY_NODEF(712), ERROR_ENTRY_NODEF(713), ERROR_ENTRY_NODEF(714),
ROR_ENTRY_NODEF(715), ERROR_ENTRY_NODEF(716), ERROR_ENTRY_NODEF(717),
ROR_ENTRY_NODEF(718), ERROR_ENTRY_NODEF(719),
ROR_ENTRY(720, "Некорректный целочисленный литерал (не число)"),
 далее пустые записи начиная с 730
ROR_ENTRY_NODEF10(730),
ROR_ENTRY(731, "Ошибка в условии цикла"),
ROR_ENTRY_NODEF10(740),
```

Рисунок 5.1 Перечень сообщений семантического анализатора.

На этапе семантического анализа предусмотрено возникновение 12 возможных ошибок.

5.4 Принцип обработки ошибок

Ошибки, возникающие в процессе трансляции программы, фиксируются в протокол, заданный входным параметрами. В случае возникновения ошибок происходит их протоколирование с номером ошибки и диагностическим сообщением.

5.5 Контрольный пример

Соответствие примеров некоторых ошибок в исходном коде и диагностических сообщений об ошибках приведено в таблице 5.1.

Таблица 5.1 - Примеры диагностики ошибок

Исходный код	Текст сообщения

main{
int var x = 15;
x = x + “string”;
}
	Ошибка на этапе Семантического анализатора
	Ошибка 701: Типы данных в выражении не совпадают, строка 3

main{
int var x;
x = 5 / 0;
}
	Ошибка на этапе Семантического анализатора
	Ошибка 700: Деление на 0, строка 3, лексема 5

main{
str var y;
y = "test2" + “test3”;
}
	Ошибка на этапе Семантического анализатора
	Ошибка 702: Недопустимое строковое выражение справа от знака '=', строка 3

Общая информация про ошибки выводится непосредственно в консоль и текстовый файл.

6. Вычисление выражений

6.1 Выражения, допускаемые языком

В языке IAS-2025 допускаются вычисления выражений целочисленного типа данных с поддержкой вызова функций внутри выражений. Приоритет операций представлен на таблице 6.1.

Таблица 6.1 - Приоритеты операций

Операция	Значение приоритета
%	4
/	4
*	4
-	3
+	3
,	2
( )	1
<<	2
>>	2

6.2 Польская запись и принцип её построения

Все выражения языка IAS-2025 преобразовываются к обратной польской записи.

Польская запись – это альтернативный способ записи арифметических выражений, преимущество которого состоит в отсутствии скобок. Существует два типа польской записи: прямая и обратная, также известные как префиксная и постфиксная. Отличие их от классического, инфиксного способа заключается в том, что знаки операций пишутся не между, а, соответственно, до или после аргументов. Алгоритм построения польской записи:

– исходная строка: выражение;
– результирующая строка: польская запись;
– стек: пустой;
– исходная строка просматривается слева направо;
– операнды переносятся в результирующую строку;
– операция записывается в стек, если стек пуст;
– операция выталкивает все операции с большим или равным приоритетом в результирующую строку;
– отрывающая скобка помещается в стек;
– закрывающая скобка выталкивает все операции до открывающей скобки, после чего обе скобки уничтожаются.

6.3 Программная реализация обработки выражений

Программная реализация алгоритма преобразования выражений к польской записи представлена в приложении Г.

6.4 Контрольный пример

Пример преобразования выражений из контрольных примеров к обратной польской записи представлен в таблице 6.2.

Таблица 6.2 - Преобразование выражений к ПОЛИЗ

Выражение	Обратная польская запись для выражения
l	l
i#l	il#
i#l	il#

ПОЛИЗ, или, как её еще называют, обратная польская запись, это способ бесскобочного представления выражений (не только арифметических), в которых операнды предшествуют операции.

7. Генерация кода

7.1 Структура генератора кода

В языке IAS-2025 генерация кода является заключительным этапом трансляции. Генератор принимает на вход таблицы лексем и идентификаторов, полученные в результате лексического анализа. В соответствии с таблицей лексем строится выходной файл на языке ассемблера, который будет являться результатом работы транслятора. В случае возникновения ошибок генерация кода не будет осуществляться. Структура генератора кода IAS-2025 представлена на рисунке 7.1.

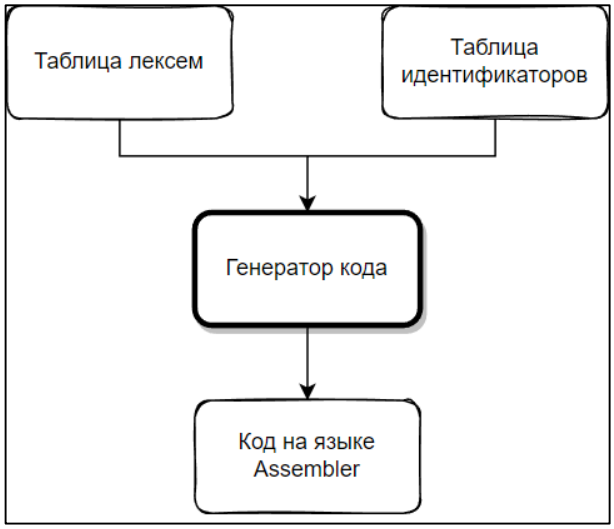

Рисунок 7.1 Структура генератора кода

Такое построение генератора кода обеспечивает последовательное и корректное преобразование промежуточного представления программы в итоговый ассемблерный код. Благодаря модульной архитектуре генератор легко адаптируется к расширению языка и добавлению новых конструкций.

7.2 Представление типов данных в оперативной памяти

Элементы таблицы идентификаторов располагаются в сегментах .data и .const языка ассемблера, что обеспечивает корректное размещение переменных и констант в памяти при выполнении программы. Соответствия между типами данных идентификаторов на языке IAS-2025 и их представлением на уровне ассемблера приведены в таблице 7.1, что позволяет однозначно интерпретировать данные при трансляции и гарантирует корректность создаваемого машинного кода.

Таблица 7.1 – Соответствия типов идентификаторов языка IAS-2025 и языка ассемблера

Тип идентификатора на языке IAS-2025	Тип идентификатора на языке ассемблера	Пояснение
int	sdword	Хранит целочисленный тип данных.
str	dword	Хранит указатель на начало строки. Строка должна завершаться символом нуль-терминатор.
char	byte	Хранит один символьный байт. При выводе формируется временная строка с нуль-терминатором.
bool	byte	Хранит логическое значение: 0 – false, 1 – true. При выводе отображается как "true" или "false".

В процессе генерации кода каждому идентификатору сопоставляется соответствующая директива ассемблера, определяющая его тип, область хранения и начальное значение.

7.3 Статическая библиотека

В языке IAS-2025 предусмотрена статическая библиотека. Статическая библиотека содержит функции, написанные на языке C++. Объявление функций статической библиотеки генерируется автоматически в коде ассемблера.

Таблица 7.3 – Функции статической библиотеки

Функция	Назначение
void write_str(char* str)	Вывод на консоль строки str
void write_int(int num)	Вывод на консоль целочисленной переменной num
int length (char* str)	Вычисление длины строки
char* copy (char* str1, char* str2, int count)	Копирование определенного количества символов из str2 в строку str1
int squareOfNumber(int num)	Извлекает квадратный корень из числа num
int factorialOfNumber(int num)	Вычисляет факториал заданного числа
int random(int start, int end)	Генерирует случайное значение в диапазоне от strat до end
int powNumber(int num, int num2)	Возводит числовое значение num в степень num2
char* getLocalTimeAndDate()	Возвращает текущее локальное время и дату

Продолжение таблицы 7.3

int asciiCode(char c)	Возврашает код текущего символа в кодировке ASCII

Объявление функций статической библиотеки генерируется автоматически.

7.4 Особенности алгоритма генерации кода

В языке IAS-2025 генерация кода строится на основе таблиц лексем и идентификаторов. Общая схема работы генератора кода представлена на рисунке 7.2.

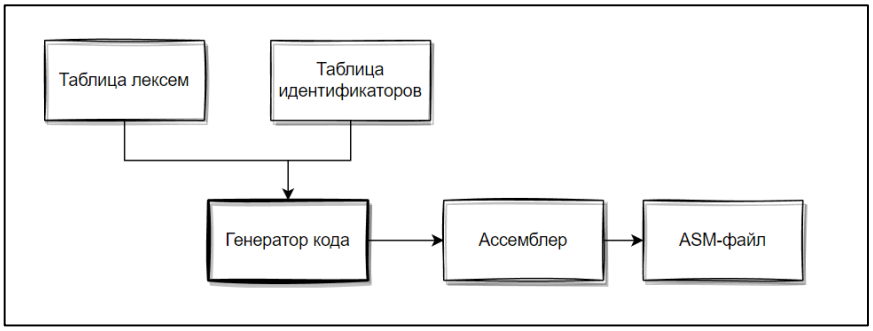

Рисунок 7.2 Структура генератора кода

7.5 Входные параметры генератора кода

На вход генератору кода поступают таблицы лексем и идентификаторов исходного код программы на языке IAS-2025. Результаты работы генератора кода выводятся в файл с расширением .asm.

7.6 Контрольный пример

Результат работы контрольного примера приведён на рисунке 7.2.

```
; --- сохранить регистры ---
push ebx
push edx
; ---------------------
push StringOne_IsStringsEquals
call lenght
push eax
pop lenOne_IsStringsEquals

push StringTwo_IsStringsEquals
call lenght
push eax
pop lenTwo_IsStringsEquals

mov edx, lenOne_IsStringsEquals
cmp edx, lenTwo_IsStringsEquals

jz true0
jnz false0
true0:
mov al, 1
mov res_IsStringsEquals, al

jmp next0

false0:
mov al, 0
mov res_IsStringsEquals, al
```

Рисунок 7.2 Результат работы программы на языке IAS-2025

Результат генерации ассемблерного кода на основе контрольного примера из приложения А приведен в приложении Д.

8. Тестирование транслятора

8.1 Тестирование проверки на допустимость символов

В языке IAS-2025 не разрешается использовать запрещённые входным алфавитом символы. Результат использования запрещённого символа показан в таблице 8.1.

Таблица 8.1 - Тестирование проверки на допустимость символов

Исходный код	Диагностическое сообщение
main { ~ }	Ошибка при чтении из входного файла Ошибка 110: Недопустимый символ в исходном файле (-in), строка 1, столбец 8

main
{
output "test
}
	Ошибка при чтении из входного файла
	Ошибка 111: Нет закрывающей кавычки (-in), строка 3, столбец 16

В ходе тестирования ошибок выявлено не было

8.2 Тестирование лексического анализатора

На этапе лексического анализа в языке IAS-2025 могут возникнуть ошибки, описанные в пункте 3.7. Результаты тестирования лексического анализатора показаны в таблице 8.2.

Таблица 8.2 - Тестирование лексического анализатора

Исходный код	Диагностическое сообщение

main
{
test123
}
	Ошибка на этапе лексического анализатора
	Ошибка 120: Нераспознанная лексема, строка 3, лексема 1

main
{
var test;
}
	Ошибка на этапе лексического анализатора
	Ошибка 121: Тип данных идентификатора не определен, строка 3, лексема 2

main
{
int var test;
int var test;
}
	Ошибка на этапе лексического анализатора
	Ошибка 131: Переопределение, строка 4, лексема 3

Продолжение таблицы 8.2

main
{
int test;
}
	Ошибка на этапе лексического анализатора. Ошибка 132: Тип идентификатора не определен, строка 3, лексема 2

int function FindMaxLen(str param x, str param y){}	Ошибка 133: Отсутствует точка входа main

main {}
main
{
int var test;
}
	Ошибка 134: Обнаружено несколько точек входа main

После анализа представленных результатов можно сделать вывод, что лексический анализатор корректно обрабатывает как типичные, так и ошибочные входные данные, обеспечивая надёжную фильтрацию некорректных конструкций на раннем этапе трансляции.

8.3 Тестирование синтаксического анализатора

На этапе синтаксического анализа в языке IAS-2025 могут возникнуть ошибки, описанные в пункте 4.6. Результаты тестирования синтаксического анализатора показаны в таблице 8.3.

Таблица 8.3 - Тестирование синтаксического анализатора

Исходный код	Диагностическое сообщение

function main
{
int var test;
}
	600: строка 1, Неверная структура программы

main
{
return 1;
}
	601: строка 4, Ошибочный оператор

main
{
int var test;
test = 1 + - 2;
}
	602: строка 4, Ошибка в выражении

int function FindMaxLen(str param x,) { } main {}	603: строка 1, Ошибка в параметрах при определении функции

Продолжение таблицы 8.3

main
{
if (1 > 0)
then
}
	606: строка 4, Ошибка при условном переходе

main
{
int str var test;
}
	607: строка 3, Ошибка при определении переменной

main
{
if {1 > 0)
then {}
}
	608: строка 3, Ошибка при определении условия перехода

int function FindMax(int param x) { return 1;} main { int var res; res = FindMax(1,)}	602: строка 6, Ошибка в выражении

После анализа представленных данных можно отметить, что синтаксический анализатор корректно обрабатывает как валидные конструкции, так и различные типы ошибок, обеспечивая надёжную проверку структуры программы перед переходом к этапу семантического анализа.

8.4 Тестирование семантического анализатора

Итоги тестирования семантического анализатора на корректное обнаружение семантических ошибок приведены в таблице 8.4.

Таблица 8.4 - Тестирование семантического анализатора

Исходный код	Текст сообщения

main{
int var x = 125;
x = x + “IAS2025”;
}
	Ошибка на этапе Семантического анализатора
	Ошибка 701: Типы данных в выражении не совпадают, строка 3

main{
int var x;
x = 5 / 0;
}
	Ошибка на этапе Семантического анализатора
	Ошибка 700: Деление на 0, строка 3, лексема 5

main{ str var y;	Ошибка на этапе Семантического анализатора

Продолжение таблицы 8.4

y = "test2" + “test3”; }	Ошибка на этапе Семантического анализатора Ошибка 702: Недопустимое строковое выражение справа от знака '=', строка 3

str function Test(int param x)
{
return 0;
}
main {}
	Ошибка на этапе Семантического анализатора
	Ошибка 703: Тип функции и возвращаемое значение не совпадают, строка 3

int function Test(int param x)
{return 0;}
main
{
int var temp;
temp = Test("test");
}
	Ошибка на этапе Семантического анализатора
	Ошибка 704: Несовпадение типов передаваемых параметров, строка 6, лексема 5

int function Test(int param x)
{return 0;}
main
{
int var temp;
temp = Test(1, 2);
}
	Ошибка на этапе Семантического анализатора
	Ошибка 705: Кол-во ожидаемых функцией и передаваемых параметров не совпадают, строка 6

main
{
if ("test1" > "test2")
then{}
}
	Ошибка на этапе Семантического анализатора
	Ошибка 706: Неверное условное выражение, строка 3

Семантический анализ в языке IAS-2025 содержит множество проверок по семантическим правилам, описанным в пункте 1.16.

Заключение

В ходе выполнения курсовой работы был разработан транслятор и генератор кода для языка программирования IAS-2025 со всеми необходимыми компонентами. Таким образом, были выполнены основные задачи данной курсовой работы:

– сформулирована спецификация языка IAS-2025;

– разработаны конечные автоматы и важные алгоритмы на их основе для эффективной работы лексического анализатора;

– осуществлена программная реализация лексического анализатора, распознающего допустимые цепочки спроектированного языка;

– разработана контекстно-свободная, приведённая к нормальной форме Грейбах, грамматика для описания синтаксически верных конструкций языка;

– осуществлена программная реализация синтаксического анализатора;

– разработан семантический анализатор, осуществляющий проверку используемых инструкций на соответствие логическим правилам;

– разработан транслятор кода на язык ассемблера;

– проведено тестирование всех вышеперечисленных компонентов.

Окончательная версия языка IAS-2025 включает:

– 4 типа данных;

– поддержка оператора вывода;

– возможность вызова 9 функций стандартной библиотеки;

– наличие 5 арифметических операторов для вычисления выражений;

– поддержка функций, операторов цикла и условия;

–Структурированная и классифицированная система для обработки ошибок пользователя.

Проделанная работа позволила получить необходимое представление о структурах и процессах, использующихся при построении трансляторов, а также основные различия и преимущества тех или иных средств трансляции.

Список использованных источников

1. Ахо, А. Компиляторы: принципы, технологии и инструменты / А. Ахо, Р. Сети, Дж. Ульман. – М.: Вильямс, 2003. – 768с.

2. Ахо, А. Теория синтаксического анализа, перевода и компиляции /А. Ахо, Дж. Ульман. – Москва : Мир, 1998. – Т. 2 : Компиляция. - 487 с.

3. Герберт, Ш. Справочник программиста по C/C++ / Шилдт Герберт. - 3-е изд. – Москва : Вильямс, 2003. - 429 с.

4. Прата, С. Язык программирования C++. Лекции и упражнения / С. Прата. – М., 2006 — 1104 с.

5. Страуструп, Б. Принципы и практика использования C++ / Б. Страуструп – 2009 – 1238 с

Приложение А

```
int function get_sum(int param argOne, int param argTwo){
     int var res = argOne + argTwo;
     return res;
}

bool function IsStringsEquals(str param StringOne, str param
StringTwo){
     bool var res;
     int var lenOne = lenght(StringOne);
     int var lenTwo = lenght(StringTwo);
     if (lenOne & lenTwo)
     then{
          res = true;
     }
     else{
          res = false;
     }
     return res;
}
bool function IsEven(int param arg){
     bool var res;
     int var buf = arg % 2;
     if (buf & 0)
     then{
          res = true;
     }
     else{
          res = false;
     }
     return res;
}
main
{
str var date = getLocalTimeAndDate();
output date;
int var a = b1010; #10
output "Переменная a";
output a;
int var b = h32; #50
output "Переменная b";
output b;
int var rnd = random(a, b);
output "Случайное число от a до b";
output rnd;
char var ch = 'c';
int var code = asciiCode(ch);
output "Код ch";
```

Листинг 1 – Исходный код программы на языке IAS-2025

```
output code;
a = a >> 2;
output "a после сдвига вправо на 2";
output a;
b = b << 2;
output "b после сдвига влево на 2";
output b;
int var iter = 0;
repeat(10){
     output iter;
     iter = iter + 1;
}
repeat(iter > 1){
     output "new iter";
     output iter;
     iter = iter - 1;
}

output "Выполнение функции get_sum(a, b)";
int var sum = get_sum(a, b);
output sum;

str var stringOne = "Hello, my name is Arseniy";
str var stringTwo = "Hello, my name is Alex";

output "Первых три элемента, скопированных из первой строки во
вторую :";
str var ns = copy(stringOne, stringTwo, 3 );
output ns;
output stringOne;
output stringTwo;

bool var flag = IsStringsEquals(stringOne,stringTwo);
output flag;
int var test = 2;
output "test:";
output test;
test = powNumber(test, 3);
output "test после powNumber:";
output test;
test = factorialOfNumber(test);
output "Факториал переменной test:";
output test;
bool var evenFlag = IsEven(test);
output evenFlag;
}
```

Листинг 1 (продолжение) – исходный код программы на языке IAS-2025

Приложение Б

ПЕРЕМЕННЫЕ

Индекс LE	Область видимости	Идентификатор	Тип Идентификатора	Тип данных	Значение	Длина строки
15	get_sum	res	V	INT	0	-
40	IsStringsEquals	res	V	BOOL	false	-
44	IsStringsEquals	lenOne	V	INT	0	-
53	IsStringsEquals	lenTwo	V	INT	0	-
95	IsEven	res	V	BOOL	false	-
99	IsEven	buf	V	INT	0	-
133	main	date	V	STR		0
144	main	a	V	INT	0	-
156	main	b	V	INT	0	-
168	main	cnd	V	INT	0	-
185	main	ch	V	CHAR	' '	-
191	main	code	V	INT	0	-
230	main	iter	V	INT	0	-
274	main	sum	V	INT	0	-
288	main	stringOne	V	STR		0
294	main	stringTwo	V	STR		0
303	main	ns	V	STR		0
325	main	flag	V	BOOL	false	-
339	main	test	V	INT	0	-
379	main	evenFlag	V	BOOL	false	-

ФУНКЦИИ

Индекс LE	Область видимости	Идентификатор	Тип данных возврата
0	global	lenght	INT
0	global	copy	STR
0	global	getLocalTimeAndDate	STR
0	global	powNumber	INT
0	global	random	INT
0	global	squareOfNumber	INT
0	global	factorialOfNumber	INT
0	global	asciiCode	INT
2	global	get_sum	INT
27	global	IsStringsEquals	BOOL
86	global	IsEven	BOOL

ЛИТЕРАЛЫ

Индекс LE	Идентификатор	Тип данных	Длина строки	Значение
70	L0	BOOL	-	true
77	L1	BOOL	-	false
103	L2	INT	-	2
109	L3	INT	-	0
146	L4	INT	-	10
149	L5	STR	14	""Переменная a""
158	L6	INT	-	50
161	L7	STR	14	""Переменная b""
178	L8	STR	27	""Случайное число от a до b""
187	L9	CHAR	-	'c'
199	L10	STR	8	""Код ch""
211	L11	STR	28	""a после сдвига вправо на 2""
223	L12	STR	27	""b после сдвига влево на 2""
246	L13	INT	-	1
257	L14	STR	10	""new iter""
270	L15	STR	34	""Выполнение функции get_sum(a, b)""
290	L16	STR	27	""Hello, my name is Arseniy""
296	L17	STR	24	""Hello, my name is Alex""
299	L18	STR	65	""Первых три элемента, скопированных из первой с..."
311	L19	INT	-	3
344	L20	STR	7	""test:""
359	L21	STR	23	""test после powNumber:""
372	L22	STR	28	""Факториал переменной test:""

Рисунок 1 – Таблица идентификаторов после контрольного примера

```
1     tfi(tpi,tpi){
2     tvi=i#i;
3     ri;
4     }
5
6     xfi(tpi,tpi){
7     xvi;
8     tvi=i(i);
9     tvi=i(i);
10    ?(i&i)
11    :{
```

```
12   i=o;
13   }
14   !{
15   i=o;
16   }
17   ri;
18   }
19   xfi(tpi){
20   xvi;
21   tvi=i#l;
22   ?(i&l)
23   :{
24   i=o;
25   }
26   !{
27   i=o;
28   }
29   ri;
30   }
31   m
32   {
33   wl;
34   tvi=i();
35   wi;
36   tvi=l;
37   wl;
38   wi;
39   tvi=l;
40   wl;
41   wi;
42   tvi=i(i,i);
43   wl;
44   wi;
45   cvi=k;
46   tvi=i(i);
47   wl;
48   wi;
49   i=inl;
50   wl;
51   wi;
52   i=ibl;
53   wl;
54   wi;
55   tvi=l;
56   ~(l){
57   wi;
58   i=i#l;
59   }
60   ~(i>l){
61   wl;
62   wi;
63   i=i#l;
```

```
64    }
65
66    wl;
67    tvi=i(i,i);
68    wi;
69
70    tvi=l;
71    tvi=l;
72
73    wl;
74    tvi=i(i,i,l);
75    wi;
76    wi;
77    wi;
78
79    xvi=i(i,i);
80    wi;
81    tvi=l;
82    wl;
83    wi;
84    i=i(i,l);
85    wl;
86    wi;
87    i=i(i);
88    wl;
89    wi;
90    xvi=i(i);
91    wi;
92    tvi=l;
93    wi;
94    }$
```

Листинг 1 – Таблица лексем для контрольного примера

Приложение В

```
          Greibach greibach(NS('S'), TS('$'), 13,
     Rule(NS('S'), GRB_ERROR_SERIES + 0, // Неверная структура
программы
          5,   // m{N}S | tfi(F){NrU;}S | cfi(F){NrU;}S | пустой
переход
          Rule::Chain(5, TS('m'), TS('{'), NS('N'), TS('}'),
NS('S')),
          Rule::Chain(13, TS('t'), TS('f'), TS('i'), TS('('),
NS('F'), TS(')'), TS('{'), NS('N'), TS('r'), NS('U'), TS(';'),
TS('}'), NS('S')),
          // Добавлено правило для функций типа char (c)
          Rule::Chain(13, TS('c'), TS('f'), TS('i'), TS('('),
NS('F'), TS(')'), TS('{'), NS('N'), TS('r'), NS('U'), TS(';'),
TS('}'), NS('S')),
          Rule::Chain(13, TS('x'), TS('f'), TS('i'), TS('('),
NS('F'), TS(')'), TS('{'), NS('N'), TS('r'), NS('U'), TS(';'),
TS('}'), NS('S')),
          Rule::Chain()
     ),
     Rule(NS('N'), GRB_ERROR_SERIES + 1, 9,
          Rule::Chain(),  // ← ПУСТОЕ ПРАВИЛО ПЕРВЫМ!
          Rule::Chain(4, TS('t'), NS('Y'), TS(';'), NS('N')),
          Rule::Chain(4, TS('c'), NS('Y'), TS(';'), NS('N')),
          Rule::Chain(5, TS('i'), TS('='), NS('E'), TS(';'),
NS('N')),
          Rule::Chain(4, TS('x'), NS('Y'), TS(';'), NS('N')),
          Rule::Chain(4, TS('w'), NS('E'), TS(';'), NS('N')),
          Rule::Chain(4, TS('~'), NS('K'), NS('Z'), NS('N')),
          Rule::Chain(6, TS('~'), TS('('), TS('l'), TS(')'),
NS('Z'), NS('N')),
          Rule::Chain(4, TS('?'), NS('K'), NS('J'), NS('N')),
          Rule::Chain(4, TS('i'), NS('M'), TS(';'), NS('N'))
     ),
     Rule(NS('E'), GRB_ERROR_SERIES + 2,         // Ошибка в
выражении
          6, // iM | lM | kM | (E)M | i(W)M
          Rule::Chain(2, TS('i'), NS('M')),
          Rule::Chain(2, TS('l'), NS('M')),
          // Добавлено для символьного литерала (k)
          Rule::Chain(2, TS('o'), NS('M')),
          Rule::Chain(2, TS('k'), NS('M')),
          Rule::Chain(4, TS('('), NS('E'), TS(')'), NS('M')),
          Rule::Chain(5, TS('i'), TS('('), NS('W'), TS(')'),
NS('M'))
     ),
     Rule(NS('M'), GRB_ERROR_SERIES + 2, 4,
          Rule::Chain(2, TS('#'), NS('E')),
          Rule::Chain(2, TS('b'), NS('E')),  // <<
Rule::Chain(2, TS('n'), NS('E')),  // >>
          Rule::Chain()
```

```
     ),
     Rule(NS('F'), GRB_ERROR_SERIES + 3,         // Ошибка в
параметрах при определении функции
          5, // tP | cP | tP,F | cP,F
          Rule::Chain(2, TS('t'), NS('P')),
          // Добавлено для параметров типа char
          Rule::Chain(2, TS('c'), NS('P')),
          Rule::Chain(4, TS('t'), NS('P'), TS(','), NS('F')),
          // Добавлено для параметров типа char (продолжение
списка)
          Rule::Chain(4, TS('c'), NS('P'), TS(','), NS('F')),
          Rule::Chain(4, TS('x'), NS('P'), TS(','), NS('F'))
     ),
     Rule(NS('P'), GRB_ERROR_SERIES + 3,         // Ошибка в
параметрах при определении функции
          1, // pi
          Rule::Chain(2, TS('p'), TS('i'))
     ),
     Rule(NS('W'), GRB_ERROR_SERIES + 4,         // Ошибка в
параметрах при вызове функции
          9, // i | l | k | i,W | l,W | k,W
          Rule::Chain(1, TS('i')),
          Rule::Chain(1, TS('l')),
          // Добавлено для char литерала (k)
          Rule::Chain(1, TS('k')),
          Rule::Chain(1, TS('o')),
          Rule::Chain(3, TS('i'), TS(','), NS('W')),
          Rule::Chain(3, TS('l'), TS(','), NS('W')),
          // Добавлено для char литерала в списке (k)
          Rule::Chain(3, TS('k'), TS(','), NS('W')),
          Rule::Chain(3, TS('o'), TS(','), NS('W')),
          Rule::Chain()
     ),
     Rule(NS('U'), GRB_ERROR_SERIES + 5,    // Только литерал
или идентификатор
          11,  // l | k | i ...
          Rule::Chain(1, TS('l')),
          // Добавлено k
          Rule::Chain(1, TS('k')),
          Rule::Chain(1, TS('i')),
          Rule::Chain(2, TS('l'), NS('U')),
          Rule::Chain(2, TS('i'), NS('U')),
          Rule::Chain(3, TS('i'), TS(','), NS('U')),
          Rule::Chain(3, TS('l'), TS(','), NS('U')),
          // Добавлено k в списки
          Rule::Chain(3, TS('k'), TS(','), NS('U')),
          Rule::Chain(2, TS('#'), TS('l')),
          // Добавлено k с оператором
          Rule::Chain(2, TS('#'), TS('k')),Rule::Chain(3,
TS('#'), TS('l'), NS('U')),
          Rule::Chain(4, TS('#'), TS('l'), TS(','), NS('U'))
     ),
```

```
     Rule(NS('J'), GRB_ERROR_SERIES + 6,     // Ошибка при
условном переходе
          2,   // :{N} | :{N}!{N}
          Rule::Chain(4, TS(':'), TS('{'), NS('N'), TS('}')),
          Rule::Chain(8, TS(':'), TS('{'), NS('N'), TS('}'),
TS('!'), TS('{'), NS('N'), TS('}'))
     ),
     Rule(NS('Y'), GRB_ERROR_SERIES + 7,     // Ошибка при
определении переменной
          5,   // vi || vi = L
          Rule::Chain(2, TS('v'), TS('i')),
          Rule::Chain(4, TS('v'), TS('i'), TS('='), NS('L')),
          Rule::Chain(5, TS('v'), TS('i'), TS('='), NS('E'),
NS('M')), // для выражений: vi = a >> 3
          Rule::Chain(4, TS('v'), TS('i'), TS('='), TS('k')),
          Rule::Chain(4, TS('v'), TS('i'), TS('='), TS('o'))
     ),

     Rule(NS('K'), GRB_ERROR_SERIES + 8,     // Ошибка при
определении условия перехода
          4,   // (U<U) | (U>U)
          Rule::Chain(5, TS('('), NS('U'), TS('<'), NS('U'),
TS(')')),
          Rule::Chain(5, TS('('), NS('U'), TS('>'), NS('U'),
TS(')')),
          Rule::Chain(5, TS('('), NS('U'), TS('&'), NS('U'),
TS(')')),
          Rule::Chain(5, TS('('), NS('U'), TS('^'), NS('U'),
TS(')'))
     ),
     Rule(NS('L'), GRB_ERROR_SERIES + 9, 18, // увеличиваем с 12
до 14
          Rule::Chain(1, TS('i')),
          Rule::Chain(1, TS('l')),
          Rule::Chain(1, TS('k')),
          Rule::Chain(1, TS('o')),
          Rule::Chain(4, TS('i'), TS('('), NS('U'), TS(')')),
          Rule::Chain(3, TS('i'), TS('('), TS(')')),
          Rule::Chain(2, TS('#'), TS('i')),
          Rule::Chain(2, TS('#'), TS('l')),
          Rule::Chain(2, TS('#'), TS('k')),
          Rule::Chain(2, TS('#'), TS('o')),
          Rule::Chain(3, TS('#'), TS('i'), NS('L')),
          Rule::Chain(3, TS('#'), TS('l'), NS('L')),
          Rule::Chain(3, TS('#'), TS('k'), NS('L')),
          Rule::Chain(3, TS('#'), TS('o'), NS('L')),
          Rule::Chain(2, TS('i'), NS('L')),
Rule::Chain(2, TS('l'), NS('L')),
          Rule::Chain(2, TS('o'), NS('L')),

          Rule::Chain(3, NS('E'), NS('M'))  // ← ДОБАВИЛИ:
выражение с оператором
```

```
     ),
     Rule(NS('Z'), GRB_ERROR_SERIES + 10,   //ошибка в теле
цикла
          1,
          Rule::Chain(3, TS('{'), NS('N'), TS('}'))
     )
);
```

Листинг 1 – Грамматика языка IAS-2025

```
     struct MFST // магазинный автомат
     {
          enum RC_STEP //код возврата функции step
          {
               NS_OK,              // найдено правило и цепочка,
цепочка записана в стек
               NS_NORULE,          // не найдено правило
грамматики (ошибка в грамматике)
               NS_NORULECHAIN,     // не найдена походящая
цепочка правила (ошибка в исходном коде)
               NS_ERROR,      // неизвесный нетерминальный символ
грамматики
               TS_OK,              // тек. символ ленты ==
вершине стека, продвинулась лента, pop стека
               TS_NOK,             // тек. символ ленты !=
вершине стека, восстановленно состояние
               LENTA_END,          // теущая позиция ленты >=
lenta_size
               SURPRISE       // неожиданный код возврата (ошибка
в step)
          };

          struct MFST_Diagnosis    // диагностика
          {
               short     posInLent;          // позиция на ленте
               RC_STEP   rc_step;            // код завершения
шага
               short     ruleNum;            // номер правила
     short     nrule_chain;        // номер цепочки правила
               MFST_Diagnosis();
               MFST_Diagnosis(short posInLent, RC_STEP rc_step,
short ruleNum, short ruleChainNum);
          } diagnosis[MFST_DIAGN_NUMBER]; // последние самые
глубокие сообщения

          GRBALPHABET* lenta;                     //
перекодированная (TS/NS) лента (из LEX)
          short currentPosInLent;                 // текущая
позиция на ленте
          short currentRule;                      // номер
текущего правила
```

```
          short currentRuleChain;                 // номер
текущей цепочки, текущего правила
          short lenta_size;                       // размер ленты
          GRB::Greibach grebach;                  // грамматика
Грейбах
          LT::LexTable lexTable;
          MFSTSTSTACK st;                              // стек
автомата
          std::stack<MFSTState> storestate;  // стек для
сохранения состояний

          MFST();
          MFST(const LT::LexTable& lexTable,GRB::Greibach
grebach);

          char* getCSt(char* buf);
     //получить содержиое стека
          char* getCLenta(char* buf, short pos, short n = 25);
     //лента: n символов, начиная с pos
          char* getDiagnosis(short n, char* buf);
     //получить n-ую строку диагностики или '\0'

          bool savestate(std::ostream* stream);   //сохранить
состояние автомата
          bool resetstate(std::ostream* stream);  //восстановить
состояние автомата
          bool push_chain(GRB::Rule::Chain chain);

          RC_STEP step(std::ostream* stream);
     //выполнить шаг автомата
          bool start(std::ostream* stream);       //запустить
автомат
          bool savedDiagnosis(RC_STEP prc_step);

          void printRules(std::ostream* stream);  //вывести
последовательность правил
struct Deducation        // вывод
          {
               short stepsCount;             // количество шагов
в выводе
               short* nRules;                // номер правила
грамматики
               short* nChainsOfRules;        // номер цепочек
правил грамматики

               Deducation()
               {
                    this->stepsCount = 0;
                    this->nRules = 0;
                    this->nChainsOfRules = 0;
               }
          }deducation;
```

```
          bool saveoutputTree();            // сохранить дерево
вывода
          };
```

Листинг 2 – структура магазинного автомата

```
struct Greibach // грамматика Грейбаха
     {
          short size;                  // количество правил
          GRBALPHABET startN;      // стартовый символ
          GRBALPHABET stbottomT;  // дно стека
          Rule* rules;             // множество правил

          Greibach(){
               size = 0;
               startN = 0;
               stbottomT = 0;
               rules = 0;};
          Greibach(GRBALPHABET pstartN, // стартовый символ
               GRBALPHABET pstbottomT,       // дно стека
               short psize,                  // количество правил
               Rule r, ...);                 // правила
          short getRule(                // получить правило
               GRBALPHABET pnn,         // левый символ правила
               Rule& prule);            // возвращаемое правило

          Rule getRule(short n);        // получить правило по
номеру

          };
```

Листинг 3 – Структура грамматики Грейбаха

```
129 : S->m{N}S
131 : N->tY;N
132 : Y->vi=L
135 : L->i()
139 : N->wE;N
140 : E->iM
141 : M->
142 : N->tY;N
143 : Y->vi=L
146 : L->l
148 : N->wE;N
149 : E->lM
150 : M->
151 : N->wE;N
152 : E->iM
153 : M->
```

```
154 : N->tY;N
155 : Y->vi=L
158 : L->l
160 : N->wE;N
161 : E->lM
162 : M->
163 : N->wE;N
164 : E->iM
165 : M->
166 : N->tY;N
167 : Y->vi=L
170 : L->i(U)
172 : U->i,U
174 : U->i
177 : N->wE;N
178 : E->lM
179 : M->
180 : N->wE;N
181 : E->iM
182 : M->
183 : N->cY;N
184 : Y->vi=L
187 : L->k
189 : N->tY;N
190 : Y->vi=L
193 : L->i(U)
195 : U->i
198 : N->wE;N
199 : E->lM
200 : M->
201 : N->wE;N
202 : E->iM
203 : M->
204 : N->i=E;N
206 : E->iM
207 : M->nE
208 : E->lM
209 : M->
210 : N->wE;N
211 : E->lM
212 : M->
213 : N->wE;N
214 : E->iM
215 : M->
216 : N->i=E;N
218 : E->iM
219 : M->bE
220 : E->lM
221 : M->
222 : N->wE;N
223 : E->lM
224 : M->
```

```
225 : N->wE;N
226 : E->iM
227 : M->
228 : N->tY;N
229 : Y->vi=L
232 : L->l
234 : N->~(l)ZN
238 : Z->{N}
239 : N->wE;N
240 : E->iM
241 : M->
242 : N->i=E;N
244 : E->iM
245 : M->#E
246 : E->lM
247 : M->
248 : N->
249 : N->~KZN
250 : K->(U>U)
251 : U->i
253 : U->l
255 : Z->{N}
256 : N->wE;N
257 : E->lM
258 : M->
259 : N->wE;N
260 : E->iM
261 : M->
262 : N->i=E;N
264 : E->iM
265 : M->#E
266 : E->lM
267 : M->
268 : N->
269 : N->wE;N
270 : E->lM
271 : M->
272 : N->tY;N
273 : Y->vi=L
276 : L->i(U)
278 : U->i,U
280 : U->i
283 : N->wE;N
284 : E->iM
285 : M->
286 : N->tY;N
287 : Y->vi=L
290 : L->l
292 : N->tY;N
293 : Y->vi=L
296 : L->l
298 : N->wE;N
```

```
299 : E->lM
300 : M->
301 : N->tY;N
302 : Y->vi=L
305 : L->i(U)
307 : U->i,U
309 : U->i,U
311 : U->l
314 : N->wE;N
315 : E->iM
316 : M->
317 : N->wE;N
318 : E->iM
319 : M->
320 : N->wE;N
321 : E->iM
322 : M->
323 : N->xY;N
324 : Y->vi=L
327 : L->i(U)
329 : U->i,U
331 : U->i
334 : N->wE;N
335 : E->iM
336 : M->
337 : N->tY;N
338 : Y->vi=L
341 : L->l
343 : N->wE;N
344 : E->lM
345 : M->
346 : N->wE;N
347 : E->iM
348 : M->
349 : N->i=E;N
351 : E->i(W)M
353 : W->i,W
355 : W->l
357 : M->
358 : N->wE;N
359 : E->lM
360 : M->
361 : N->wE;N
362 : E->iM
363 : M->
364 : N->i=E;N
366 : E->i(W)M
368 : W->i
370 : M->
371 : N->wE;N
372 : E->lM
373 : M->
```

```
374 : N->wE;N
375 : E->iM
376 : M->
377 : N->xY;N
378 : Y->vi=L
381 : L->i(U)
383 : U->i
386 : N->wE;N
387 : E->iM
388 : M->
389 : N->
390 : S->
```

Листинг 4 – Дерево разбора синтаксиеского анализатора

Приложение Г

```
.586                                ; система команд (процессор
Pentium)
.model flat, stdcall                ; модель памяти, соглашение о
вызовах
includelib kernel32.lib
includelib libucrt.lib
includelib StaticLib.lib

ExitProcess PROTO: dword      ; прототип функции для завершения
процесса Windows

EXTRN lenght: proc
EXTRN write_int: proc
EXTRN write_str : proc
EXTRN copy: proc
EXTRN getLocalTimeAndDate: proc
EXTRN random: proc
EXTRN squareOfNumber: proc
EXTRN factorialOfNumber: proc
EXTRN powNumber: proc

EXTRN asciiCode: proc

.stack 4096

.const                                  ; сегмент констант -
литералы
nulError byte 'error divided by zero', 0
nul sdword 0, 0

     L0 byte 1
     L1 byte 0
     L2 sdword 2
     L3 sdword 0
     L4 sdword 10
     L5 byte "Переменная a", 0
     L6 sdword 50
     L7 byte "Переменная b", 0
     L8 byte "Случайное число от a до b", 0
     L9 byte 'c', 0
     L10 byte "Код ch", 0
     L11 byte "a после сдвига вправо на 2", 0
     L12 byte "b после сдвига влево на 2", 0
     L13 sdword 1
     L14 byte "new iter", 0
     L15 byte "Выполнение функции get_sum(a, b)", 0
```

```
     L16 byte "Hello, my name is Arseniy", 0
     L17 byte "Hello, my name is Alex", 0
     L18 byte "Первых три элемента, скопированных из первой
строки во вторую :", 0
     L19 sdword 3
     L20 byte "test:", 0
     L21 byte "test после powNumber:", 0
     L22 byte "Факториал переменной test:", 0
.data                                   ; сегмент данных -
переменные и параметры
_true_str db 'true', 0
_false_str db 'false', 0

     res_get_sum sdword 0
     res_IsStringsEquals byte 0
lenOne_IsStringsEquals sdword 0
     lenTwo_IsStringsEquals sdword 0
     res_IsEven byte 0
     buf_IsEven sdword 0
     date_main dword ?
     a_main sdword 0
     b_main sdword 0
     rnd_main sdword 0
     ch_main byte 0
     code_main sdword 0
     iter_main sdword 0
     sum_main sdword 0
     stringOne_main dword ?
     stringTwo_main dword ?
     ns_main dword ?
     flag_main byte 0
     test_main sdword 0
     evenFlag_main byte 0
.code                                   ; сегмент кода

;----------- get_sum ------------
get_sum PROC,  argOne_get_sum : sdword, argTwo_get_sum : sdword
; --- сохранить регистры ---
push ebx
push edx
; ----------------------
push argOne_get_sum
push argTwo_get_sum
pop ecx
pop eaxadd eax, ecx

push eax
pop res_get_sum
```

```
; --- восстановить регистры ---
pop edx
pop ebx
; -------------------------
mov eax, res_get_sum
ret
get_sum ENDP
;------------------------------


;----------- IsStringsEquals ------------
IsStringsEquals PROC,    StringOne_IsStringsEquals : dword,
StringTwo_IsStringsEquals : dword
; --- сохранить регистры ---
push ebx
push edx
; ----------------------
push StringOne_IsStringsEquals
call lenght
push eax
pop lenOne_IsStringsEquals

push StringTwo_IsStringsEquals
call lenght
push eax
pop lenTwo_IsStringsEquals

mov edx, lenOne_IsStringsEquals
cmp edx, lenTwo_IsStringsEquals

jz true0
jnz false0
true0:
mov al, 1
mov res_IsStringsEquals, al
jmp next0
next0:

; --- восстановить регистры ---
pop edx
pop ebx
; -------------------------
mov al, res_IsStringsEquals
movzx eax, al
ret
IsStringsEquals ENDP
;------------------------------
```

```
;----------- IsEven ------------
IsEven PROC,   arg_IsEven : sdword
; --- сохранить регистры ---
push ebx

push edx
; ----------------------
push arg_IsEven
push L2
pop ecx
pop eax
cdq
idiv ecx
push edx
pop buf_IsEven
mov edx, buf_IsEven
cmp edx, L3
jz true1
jnz false1
true1:
mov al, 1
mov res_IsEven, al
jmp next1
false1:
mov al, 0
mov res_IsEven, al
next1:
; --- восстановить регистры ---
pop edx
pop ebx
```

Листинг 1 – Исходный код на языке ассемблера для контрольного примера